Formalisme technique et critères de preuve — AAM O94.6

Dossier d'ingénierie et d'audit expérimental

20 octobre 2026

Contents

1 Objet et statut 1

2 Hypothèses, ressources et frontières 2

3 Architecture fonctionnelle 2

4 Types de preuve et statut 2

5 Formalisation de la pertinence RAG 3

6 Formalisation du multi-question 3

7 Contrat de génération longue 3

8 Programmation compositionnelle et compilation 3

9 Observation et reproduction d'un processus 4

10 Boucle de récompense/punition 4

11 Protection des preuves et phases 4

12 État connu et travail non certifié 4

13 Commandes de référence 4

1 Objet et statut

Ce document formalise la branche O94.6, dérivée de la référence immuable O94.5. O94.6 corrige les défauts reproduits pendant les essais: pertinence RAG au niveau de l'entité, traitement multi-question, visibilité des erreurs et des générations longues, raccordement de l'observation aux décodeurs disponibles, composition exécutable et boucle de récompense documentée. Le statut `PASS_BOUNDED` signifie uniquement que les scénarios, contrats et vérificateurs identifiés ont réussi; il ne constitue pas une preuve d'intelligence générale, de validité universelle ou d'achèvement de chaque but futur.

La séparation centrale est la suivante: les données de test et leurs oracles sont indépendants du runtime; la sortie réelle est conservée avant comparaison; les échecs et délais dépassés restent dans le registre; l'adaptateur déterministe est toujours étiqueté comme adaptateur de protocole, et jamais comme LLM.

2 Hypothèses, ressources et frontières

- Les données de référence O94.5 sont gelées et conservées en lecture seule.
- Les données indépendantes O94.6 vivent sous `real_task_tests/O94_6_replay/`; les oracles sont séparés de l'arbre de code AAM.
- L'observation ne lit pas le code source de la cible lors de la capture de la facture: la cible s'exécute comme sous-processus avec entrées JSONL; seules les paires observées sont incorporées à la capsule.
- Une composition exécutée est du code produit et/ou déclenché par un compilateur; il s'exécute avec les permissions du compte courant et n'est pas un bac à sable.
- La capacité de charger un modèle dépend des poids, du tokenizer, de la configuration, des dépendances et de la politique de chargement effectivement disponibles. Les poids SmolLM2-360M de l'utilisateur n'étaient pas montés dans l'environnement de cet audit.

3 Architecture fonctionnelle

Le pipeline de requête s'exprime comme une composition de compétences et de méthodes:

1. **Entrée et validation:** créer un `TaskEnvelope`, vérifier les options, rattacher la conversation et garder la requête originelle.
2. **Décomposition:** détecter les questions coordonnées ou ponctuées, propager prudemment l'entité de contexte et créer une sous-tâche par question.
3. **Observation/RAG:** collecter les pièces autorisées, construire les vues, préserver les identifiants et récupérer des passages accompagnés de leur provenance.
4. **Routage:** choisir RAG-direct, recherche lexicale/sémantique ou génération par le modèle sélectionné; les erreurs de fournisseur sont des états explicites.
5. **Génération:** transmettre la question et, s'il existe, le contexte RAG au fournisseur choisi. Les réponses générées sont des candidats non vérifiés.
6. **Vérification:** un évaluateur séparé vérifie des contraintes calculées à partir des données d'essai; il ne préremplit pas la sortie demandée au runtime.
7. **Mémoire/récompense:** consigner les événements, la route, les ressources, les sources, les retours utilisateur et les résultats vérifiés en dimensions distinctes.
8. **Réponse:** restituer la réponse ou un statut non résolu/error/timeout, avec sources et métadonnées de traçabilité.

4 Types de preuve et statut

Statut	Sens dans ce dossier	Ne prouve pas
`PASS_BOUNDED`	Les assertions déclarées du scénario ont réussi dans le budget et l'environnement consignés.	La vérité de toute génération ou une généralisation universelle.

UNRESOLVED	Aucune réponse de capsule/candidat n'a été fournie pour cette entrée, ou aucune preuve pertinente n'a été retrouvée.	Un échec permanent du modèle.
ERROR	Un fournisseur, un décodeur, une compilation ou un contrat a remonté une erreur.	Un résultat de tâche réussi.
TIMEOUT	Le budget de temps s'est épuisé avant une fin observée.	Une réponse réussie ou une impossibilité théorique.
BLOCKED	Un prérequis, une autorisation ou un contrôle de phase a empêché l'exécution.	Un test fonctionnel exécuté.

5 Formalisation de la pertinence RAG

Un candidat RAG doit être évalué en tenant compte de l'entité demandée, de l'attribut et de l'intention. Si la requête mentionne un identifiant structuré, un candidat attaché à un autre identifiant ne doit pas devenir réponse simplement parce que son vocabulaire commun est proche. Les enregistrements ligne-par-ligne sont segmentés avant classement. Les résultats possèdent source, hash, index de chunk, score et composants de score. Un identifiant inconnu doit pouvoir aboutir à `UNRESOLVED` plutôt qu'à un voisin arbitraire.

L'oracle d'évaluation est séparé. Pour l'expérience GB-42/FR-27, les sept prompts historiques O94.5 sont rejoués: la campagne O94.5 avait donné de mauvais faits d'entité, tandis que le rejeu O94.6 a obtenu 7/7 réponses attendues depuis les sources correctes. Le rapport `RAPPORT_TESTS_O94_6_FR.md` contient les prompts, réponses et références.

6 Formalisation du multi-question

Pour une requête contenant n sous-questions reconnues, le routeur crée n tâches enfants, chacune avec son texte, ses preuves et son statut. Le résultat est ordonné et ne doit pas perdre une sous-question silencieusement. Les références pronominales peuvent reprendre une entité antérieure lorsqu'elle est identifiée sans ambiguïté; chaque réponse reste évaluable séparément.

7 Contrat de génération longue

La génération asynchrone a quatre états publics: démarrée, en cours, terminée, erreur/timeout. Une terminaison valide requiert que la sortie finale soit remise au transcript et non simplement renvoyée par le fournisseur. Pour les fournisseurs réseau, un délai de socket/réponse doit être explicite. Pour une extension Python en-process, on ne prétend pas interrompre arbitrairement son code de façon sûre; le guide demande un worker isolé si une annulation forte est requise.

Un essai de fixture de protocole a rendu 58 199 caractères en 3,1804 secondes; le hash de la sortie est `fa0c90e8de700034df9f1a935864e6cbee1401c5f436cafe5e30661528e514fd`. Les marqueurs de début et de fin ont été détectés dans le transcript visible. Ce résultat valide la conservation des grandes sorties de ce scénario, pas la qualité d'un vrai LLM.

8 Programmation compositionnelle et compilation

Une composition est un graphe acyclique d'invocations, de séquences, de conditions, de collections et de références nommées, lié à un catalogue de briques. Le validateur vérifie le schéma et les identifiants de composants. Le compilateur produit du Python exécutable et le bytecode CPython; l'exécution est suivie du code de retour, de stdout, de stderr, de tests et de hashes. Le mode

interprété et le mode compilé doivent être comparés sur des copies propres pour éviter que des fichiers générés ne biaisent l'inventaire.

La composition de maintenance sous `examples/compositions/O94_6_project_quality_gate.json` comprend dix nœuds; le projet métier indépendant contient quatre actifs et produit les actions `HOLD_DISPATCH`, `AUDIT_OVERDUE`, `AUDIT_DUE` et `MONITOR`. La composition peut être copiée, modifiée et recompilée. Le reçu `real_task_tests/O94_6_replay/evidence/composed_editor_build_test.json` fournit les hashes de source Python, de bytecode et de sortie. La démonstration n'établit pas que le code métier a été inventé spontanément par le modèle.

9 Observation et reproduction d'un processus

La preuve de facture exécute une cible externe sans importer son code dans le moteur de capture. Le corpus possède huit cas: six observés pour l'appariement exact, deux gardés à l'écart. La capsule autonome rejoue les six couples d'entraînement. Les deux entrées tenues à l'écart restent `UNRESOLVED`; cela mesure explicitement l'absence de généralisation de cette capsule exacte. Le chemin reproductible pour l'utilisateur est décrit dans `real_task_tests/O94_6_replay/observed_process/README_REPRODUCTION_SUR_PC.md`.

L'environnement distant n'a pas accès au PC personnel de l'utilisateur. L'observation de cette cible a eu lieu dans l'environnement d'audit; le script de capture doit être lancé sur le PC de l'utilisateur pour observer son vrai processus. Les traces, le checkpoint et la capsule sont fournis pour audit et répétition locale.

10 Boucle de récompense/punition

La politique sépare au moins trois dimensions: préférence/retour utilisateur, fiabilité d'exécution et résultat vérifié. Une erreur opérationnelle réduit la fiabilité; une réponse marquée non vérifiée ne met pas à jour la vérité; un événement répété ne doit pas être appliqué deux fois. Un signal positif n'est produit que par une évaluation post-résolution documentée. La boucle choisit des stratégies selon les historiques et événements enregistrés mais n'est pas, à elle seule, une preuve d'intelligence générale émergente.

11 Protection des preuves et phases

Les versions précédentes sont conservées; la checklist O94.6 décrit les phases P0–P8 et leurs critères. Les sources immuables du parent O94.5 sont validées séparément de l'overlay O94.6. Les logs et reçus mutables sont authentifiés par des manifestes/reçus séparés pour éviter qu'une exécution ne se bloque en modifiant le manifeste source.

12 État connu et travail non certifié

La campagne a testé les parcours décrits ici, mais il reste des assertions plus fortes à éprouver avec un vrai fournisseur LLM et les poids de l'utilisateur. Aucun poids SmolLM2-360M n'a été disponible dans l'environnement de test; il n'est pas prétendu que ces poids ont été chargés. Le moteur de reproduction de facture réussit le rejeu exact observé mais ne généralise pas aux deux entrées cachées. Les outils externes OCR/FFmpeg/FFprobe dépendent de leur présence réelle et de leurs versions sur la machine hôte.

13 Commandes de référence

À exécuter depuis la racine du modèle:

```
python scripts/preflight_o946.py
python scripts/o946_replay_o945_regression.py --project-root . --experiment-dir real_task_tests
python scripts/o946_run_composition_quality_gate.py \
  --project-root . \
  --workspace examples/compositions/094_6_quality_gate_demo \
  --variant-limit 2
python scripts/o946_observe_invoice_pipeline.py
python scripts/o946_capture_capital_chat_evidence.py
python scripts/o946_capture_long_generation_evidence.py
```

Les essais des deux derniers scripts utilisent des fournisseurs de protocole dédiés; ils ne doivent pas être décrits comme une génération par LLM.